

VNU-HCM University of Information Technology

ASSIGNMENT 2

By Chu Cuong - 24520236

Email: cuongchu200@gmail.com
4-7-2025

MỤC LỤC

Lời mở đầu	2
1. Basic	3
1.1. Task	3
1.2. Point3D.....	5
1.3. an.512.Trộn 2 mảng.....	7
1.4. Find MEX.....	9
1.5. Point2D (template)	11
1.6. VU33_MaxStr	13
1.7. VQ44_FLOWERS(template).....	15
1.8. Kiểm kê	18
1.9. MergeSort.....	20
1.10. InsertionSort	22
1.11. BubbleSort.....	23
1.12. SelectionSort	25
2. Advance.....	26
2.1. MaxMinSum.....	26
2.2. khangtd.Login1	28
2.3. khangtd.Login2	29
2.4. khangtd.DetectVirus	29
2.5. khangtd.DetectVirus2.....	30
2.6. Linear Search 4.....	32
2.7. Vượt mức Pickleball v2	34
2.8. Bốn ông điền	36
2.9. Huấn luyện chuột	38

Lời mở đầu

Bài tập tuần này được chia thành 2 mục **Advance** và **Basic**. Tương ứng với nó là mức độ khó dễ cũng được phân hóa rõ ràng để người làm có thể ôn lại và học thêm nhiều kiến thức mới. Và sau đây là ý tưởng cũng như cách giải các bài tập được đưa ra

[IT003.P21.CTTN.2] Assignment 2 (Basic) 			[IT003.P21.CTTN.2] Assignment2 (Advance) 		
12 problems with a total score of 1200			9 problems with a total score of 900		
#	PROBLEM	SCORE	#	PROBLEM	SCORE
1	Task	100	1	MaxMinSum	100
2	Point3D	100	2	khangtd.Login1	100
3	an.512.Trộn 2 mảng	100	3	khangtd.Login2	100
4	Find.MEX	100	4	khangtd.DetectVirus2	100
5	Point2D (template)	100	5	khangtd.DetectVirus	100
6	VU33_MaxStr	100	6	Linear Search 4	100
7	VQ44_FLOWERS(template)	100	7	Vượt mức Pickleball v2 (time limit: 0.5s)	100
8	Kiểm Kê	100	8	Bốn ông điên	100
9	MergeSort	100	9	Huấn luyện chuột	100
10	InsertionSort	100			
11	BubbleSort	100			
12	SelectionSort	100			

1. Basic

1.1. Task

➤ Đề bài:

BÀI KIỂM TRA

Alice và Bob là đôi bạn thân trong lớp và thường giúp nhau trong giờ học. Hôm nay sẽ có tiết kiểm tra toán và như mọi khi – sẽ có k đề khác nhau. Lớp học có n học sinh. Các bàn trong lớp xếp thành một hàng dọc, mỗi bàn 2 người ngồi. Vị trí bên trái của bàn đánh số là 1, vị trí bên phải – đánh số 2. Đề bài được phát bắt đầu từ vị trí 1 của bàn 1 rồi đến vị trí 2 của bàn 1, tiếp theo là vị trí 1 của bàn 2, rồi tới vị trí 2 của bàn 2, . . . lần lượt các đề 1, đề 2, đề 3, . . . cho tới đề k rồi lại quay lại đề 1, đề 2, . . . cho tới khi tất cả các học sinh đều nhận được đề.

Alice đến trước và ngồi vào vị trí quen thuộc của mình: hàng p và vị trí q . Bob đến sau và muốn ngồi vào nơi sẽ nhận được cùng đề với Alice và ở đây bàn gần với dãy bàn của Alice nhất. Nếu có 2 cách ngồi thì Bob sẽ chọn bàn ở phía trước của Alice.

Hãy xác định bàn u và vị trí v trong bàn nơi Bob cần phải ngồi. Nếu không có cách ngồi thỏa mãn thì đưa ra một số -1.

Dữ liệu: Vào từ thiết bị nhập chuẩn:

- Dòng đầu tiên chứa một số nguyên n ($2 \leq n \leq 10^9$),
- Dòng thứ 2 chứa số nguyên k ($2 \leq k \leq n$),
- Dòng thứ 3 chứa số nguyên p ($1 \leq p \leq (n+1)/2$),
- Dòng thứ 4 chứa số nguyên q ($1 \leq q \leq 2$).

Kết quả: Đưa ra thiết bị xuất chuẩn một dòng 2 số nguyên u và v tìm được hoặc số -1 nếu không có cách ngồi thỏa mãn điều kiện đã nêu.

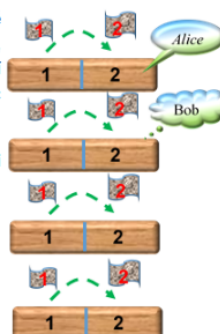
Ví dụ:

INPUT	OUTPUT
25	
2	2 2
1	
2	

➤ Ý tưởng:

Ở bài này yêu cầu ta đưa ra vị trí thỏa mãn sao cho Alice và Bob chung đề, khi đó ta nghĩ tới việc đánh số thứ tự của bàn và vị trí cộng với việc chỉ cần xét hai vị trí gần nhất mà trùng đề ở trước và sau của Alice là đủ để biết được vị trí Bob ngồi hay có cách ngồi thỏa mãn hay không. Cụ thể hơn:

- Gọi bàn của Alice nằm tại vị trí (p,q) và ta sẽ chuyển các vị trí thành 1 dãy liên tục.
- Khi đó, (p,q) sẽ được là $2 \cdot (p - 1) + q$, đặt là Alice.
- Lúc này, về mức độ ưu tiên, ta sẽ xét bàn của Bob nằm thứ Alice - $k \geq 0$ trước và Alice + $k \leq n$ rồi truy ngược ra bàn và vị trí của Bob. Nếu cả hai cái đều không thỏa thì trả về giá trị -1.



➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    long long n, k, p, q;
    cin >> n >> k >> p >> q;
    // Biến thành số thứ tự của mảng 1 chiều
    long long alice = 2 * (p - 1) + q;
    long long bob = alice - k;
    // Ưu tiên bàn phía trước của Alice
    // Ta chỉ xét hai bàn cùng đề phía trước và phía sau
    if (bob >= 1)
    {
        long long pos = bob % 2;
        long long u = bob / 2;
        if (pos == 0)
            cout << u << " " << 2;
        else
            cout << u + 1 << " " << 1;
    }
    else
    {
        bob = alice + k;
        if (bob <= n)
        {
            long long pos = bob % 2;
            long long u = bob / 2;
            if (pos == 0)
                cout << u << " " << 2;
            else
                cout << u + 1 << " " << 1;
        }
        else
            cout << -1;
    }
    return 0;
}
```

1.2. Point3D

➤ Đề bài:

Cho một tập N điểm (x, y, z) trên không gian 3 chiều Oxyz. Bạn hãy sắp xếp và in ra các điểm **tăng dần theo x** , nếu x bằng nhau thì sắp xếp các điểm **giảm dần theo y** , nếu y bằng nhau thì sắp xếp các điểm **tăng dần theo z** .

Input:

- Dòng đầu tiên chứa số nguyên N ($1 \leq N \leq 10^6$) là số lượng điểm trên không gian 3 chiều.
- N dòng tiếp theo gồm 3 số nguyên x, y, z ($-10^9 \leq x, y, z \leq 10^9$) là tọa độ của các điểm trên không gian 3 chiều.

Output: Các điểm đã được sắp xếp, mỗi điểm được in trên một hàng.

Ví dụ:

Input	Output
8	0 1 -1
1 2 1	0 0 0
1 3 2	1 3 2
2 4 3	1 2 1
2 4 1	2 4 1
4 1 -1	2 4 3
0 1 -1	4 1 -1
0 0 0	4 1 2
4 1 2	

Lưu ý: Sử dụng '\n' để xuống dòng thay vì endl để tránh bị Time Limit Exceeded (TLE)

➤ Ý tưởng:

Sau khi xem phần Code Editor thì ta biết được là bài này cần ta dùng 1 số hàm sort có sẵn như: **sort**, **stable_sort**, **partial_sort**, **partial_sort_copy**, **sort_heap**

Bài này cũng đơn giản là sắp xếp điểm ở 3 chiều theo điều kiện có sẵn, vì vậy ta chỉ cần viết hàm **sort** sử dụng quick sort cùng với **comparator** là giải được rồi.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second
int partition(vector<pair<int, pair<int, int>>> &arr, int low, int high, bool
(*comp)(pair<int, pair<int, int>>, pair<int, pair<int, int>>)){
    pair<int, pair<int, int>> pivot = arr[low];
    int i = low - 1, j = high + 1;
    while (true)
    {
        do
        {
            i++;
        } while (comp(arr[i], pivot));
        do
        {
            j--;
        } while (comp(pivot, arr[j]));

        if (i >= j)
            return j;

        swap(arr[i], arr[j]);
    }
}

void QuickSort(vector<pair<int, pair<int, int>>> &arr, int low, int high, bool
(*comp)(pair<int, pair<int, int>>, pair<int, pair<int, int>>)){
    if (low < high)
    {
        int pi = partition(arr, low, high, comp);
        QuickSort(arr, low, pi, comp);
        QuickSort(arr, pi + 1, high, comp);
    }
}

bool comp(pair<int, pair<int, int>> a, pair<int, pair<int, int>> b){
    // Nếu hoành độ khác nhau thì sắp cho nó tăng dần
    if (a.fi != b.fi)
        return a.fi < b.fi;
    // Nếu hoành độ bằng nhau thì sắp tung độ giảm dần
    else if (a.se.fi != b.se.fi)
        return a.se.fi > b.se.fi;
    // Nếu tung độ bằng nhau thì sắp cao độ tăng dần
    else
```

```

        return a.se.se < b.se.se;
    }
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;

    vector<pair<int, pair<int, int>>> v;
    for (int i = 0; i < n; i++)
    {
        int temp1, temp2, temp3;
        cin >> temp1 >> temp2 >> temp3;
        v.push_back({temp1, {temp2, temp3}});
    }
    QuickSort(v, 0, n - 1, comp);
    for (auto i : v)
        cout << i.fi << " " << i.se.fi << " " << i.se.se << '\n';
    return 0;
}

```

1.3. an.512.Trộn 2 mảng

➤ Đề bài:

Cho hai mảng đã có thứ tự, hãy trộn chúng thành một mảng cũng có thứ tự.

INPUT

Dòng đầu tiên chứa số T (T < 10) là số test case.

Mỗi test case có 3 hàng. Hàng đầu tiên chứa hai số n và m (n,m < 500.000) lần lượt là số phần tử của hai mảng A và B. Hàng thứ hai chứa n số nguyên được sắp theo thứ tự không giảm, đây chính là các phần tử của mảng A. Hàng cuối cùng chứa m số nguyên được sắp theo thứ tự không giảm, đây chính là các phần tử của mảng B.

OUTPUT

Ứng với mỗi test case, xuất ra trên một dòng mảng có thứ tự thu được bằng cách trộn hai mảng A và B

VÍ DỤ

INPUT	OUTPUT
1 5 4 1 2 5 7 10 2 4 9 11	1 2 2 4 5 7 9 10 11
2 95 21 2 3 3 3 3 3 5 6 7 8 9 10 12 14 15 17 19 21 22 23 23 24 24 26 28 29 30 30 31 33 33 35 37 38 38 38 39 39 39 39 39 39 40 40 40 42 43 45 46 48 48 50 51 52 54 55 57 59 59 60 60 61 63 64 64 66 68 70 72 72 74 75 75 76 76 76 77 79 81 82 83 83 83 84 86 88 90 91 93 94 0 0 2 2 3 4 4 5 7 7 9 9 11 11 11 13 14 16 17 19 20 65 26 2 3 5 6 7 9 9 9 11 13 14 14 15 16 17 19 19 20 21 21 22 22 24 26 28 28 29 29 29 30 30 32 32 32 34 35 36 38 39 40 41 42 42 43 44 46 47 49 50 50 51 53 54 54 54 56 57 57 58 58 60 62 64 66 66 0 0 2 4 6 6 6 8 8 9 9 11 12 13 15 16 18 20 21 21 23 23 25 26 26 26	0 0 2 2 2 3 3 3 3 3 3 4 4 5 5 6 7 7 7 8 9 9 9 10 11 11 11 12 13 14 14 15 16 17 17 19 19 20 21 22 23 23 24 24 26 28 29 30 30 31 33 33 35 37 38 38 38 39 39 39 39 39 40 40 40 42 43 45 46 48 48 50 51 52 54 55 57 59 59 60 60 61 63 64 64 66 68 70 72 72 74 75 75 76 76 76 76 77 79 81 82 83 83 83 83 84 86 88 90 91 93 94 0 0 2 2 3 4 5 6 6 6 7 8 8 9 9 9 9 11 11 12 13 13 14 14 15 15 16 16 17 18 19 19 20 20 21 21 21 21 22 22 23 23 24 25 26 26 26 26 28 28 29 29 29 30 30 32 32 32 34 35 36 38 39 40 41 42 42 43 44 46 47 49 50 50 51 53 54 54 54 56 57 57 58 58 60 62 64 66 66

➤ Ý tưởng:

Ở phần Code Editor, cho ta biết rằng chỉ cần phải viết hàm **merge** để trộn hai mảng có thứ tự thành 1, còn khai báo và hàm main đã được viết sẵn.

➤ Script:

```
#include <iostream>
using std::cin;
using std::cout;
using std::endl;

\\ Phần cần viết
void merge(int a[], int n, int b[], int m, int c[]){
    int i = 0, j = 0, k = 0;
    while(i < n && j < m){
        if(a[i] <= b[j]) c[k++] = a[i++];
        else c[k++] = b[j++];
    }
    while (i < n) c[k++] = a[i++];
    while (j < m) c[k++] = b[j++];
}
\\ Xong

int main (){
    cin.tie(0);
    std::ios::sync_with_stdio(false);
    int n, m;
    int t; cin >> t;
    while (cin >> n >> m || t > 0){
        t--;
        int *a = new int [n], *b = new int[m], *c = new int[n+m];
        for (int i = 0; i < n; i++) cin >> a[i];
        for (int i = 0; i < m; i++) cin >> b[i];

        merge(a, n, b, m, c);
        int nc = n + m;
        for(int i = 0; i < nc; i++){
            cout << c[i] << " ";
        }
        delete c;
        cout << endl;
    }
}
```

1.4. Find MEX

➤ Đề bài:

MEX (minimum excluded) là một giá trị được sử dụng phổ biến trong Lý thuyết Trò chơi. MEX được định nghĩa là số tự nhiên nhỏ nhất chưa xuất hiện trong mảng.

Ví dụ:

- $\text{MEX}([\])$ = 0.
- $\text{MEX}([0, 2, 3])$ = 1
- $\text{MEX}([0, 1, 2, 3])$ = 4.

Bạn hãy xác định MEX của một mảng cho trước?

Input:

- Dòng đầu tiên chứa số nguyên N ($1 \leq N \leq 10^5$), là số lượng phần tử của mảng.
- Dòng tiếp theo chứa N số nguyên a_i ($0 \leq a_i \leq 10^9$) là các phần tử của mảng.

Output:

- Một số nguyên là giá trị MEX của dãy được cho.

Input	Output
6 0 1 2 4 6 7	3

➤ Ý tưởng:

Thử thách này cũng không cho ta dùng các hàm **sort** phổ biến có sẵn cùng hàm **set**, **map**,... Với việc phải tìm MEX, tôi nghĩ phải sắp xếp lại rồi lại duyệt một lần nữa từ đầu đến cuối mảng để xem MEX là số nào thì sẽ chắc chắn được kết quả đúng.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<int> &arr, int low, int high)
{
    int pivot = arr[low];
    int i = low - 1, j = high + 1;

    while (true){
        // Tìm phần tử bên trái ngoài cùng của dãy
        // mà lớn hơn hoặc bằng pivot
        do
        {
            i++;
        } while (arr[i] < pivot);
```

```

        // Tìm phần tử bên phải ngoài cùng của dãy
        // mà bé hơn hoặc bằng pivot
        do
        {
            j--;
        } while (arr[j] > pivot);
        /// Nếu hai phần tử đó gặp nhau
        if (i >= j)
            return j;
        // Nếu không thì sắp xếp cho đúng
        swap(arr[i], arr[j]);
    }
}

void QuickSort(vector<int> &arr, int low, int high){
    if (low < high){
        // pi là chỉ số pivot trước đã được
        // đặt ở đúng vị trí
        int pi = partition(arr, low, high);
        // Độc lập thực hiện sắp xếp cho hai phần
        // trái và phải của pivot như vậy
        QuickSort(arr, low, pi);
        QuickSort(arr, pi + 1, high);
    }
}

int main(){
    int n;
    cin >> n;
    vector<int> a;
    for (int i = 0; i < n; i++){
        int temp;
        cin >> temp;
        a.push_back(temp);
    }
    QuickSort(a, 0, n - 1);
    // Tìm giá trị MEX của dãy sắp xếp
    int mex = 0;
    for (int i = 0; i < n; i++){
        if (a[i] == mex)
            mex++;
    }
    cout << mex;
    return 0;
}

```

1.5. Point2D (template)

➤ Đề bài:

Tập Điểm

Cho một tập N điểm (x,y) trên mặt phẳng Oxy. Bạn hãy sắp xếp và in ra các điểm tăng dần theo x , nếu x bằng nhau thì sắp xếp các điểm giảm dần theo y .

Input:

- Dòng đầu tiên chứa số nguyên N ($1 \leq N \leq 10^6$) là số lượng điểm trên mặt phẳng.
- N dòng tiếp theo gồm 2 số nguyên x, y ($-10^9 \leq x, y \leq 10^9$) là tọa độ của các điểm trên mặt phẳng.

Output: Các điểm đã được sắp xếp, mỗi điểm được in trên một hàng.

Ví dụ:

Input	Output
5	1 3
1 2	1 2
1 3	2 9
2 4	2 4
2 9	4 1
4 1	

Lưu ý: Sử dụng '\n' để xuống dòng thay vì endl để tránh bị Time Limit Exceeded (TLE)

➤ Ý tưởng:

Bài này chỉ khác bài **Point2D** của **Assignment1** ở chỗ là không được sử dụng hàm sort có sẵn của C++ thôi, còn lại thì cũng ý tưởng tương tự vậy nên ta chỉ cần viết hàm **sort**, **comparator** ra là xong.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

#define fi first
#define se second

int partition(vector<pair<int, int>> &arr, int low, int high, bool
(*comp)(pair<int, int>, pair<int, int>)){
    pair<int, int> pivot = arr[low];
    int i = low - 1, j = high + 1;
```

```

while (true){
    do{
        i++;
    } while (comp(arr[i], pivot));
    do{
        j--;
    } while (comp(pivot, arr[j]));

    if (i >= j)
        return j;
    swap(arr[i], arr[j]);
}
}

void QuickSort(vector<pair<int, int>> &arr, int low, int high, bool
(*comp)(pair<int, int>, pair<int, int>)){
    if (low < high)
        int pi = partition(arr, low, high, comp);
        QuickSort(arr, low, pi, comp);
        QuickSort(arr, pi + 1, high, comp);
    }
}

bool comp(pair<int, int> a, pair<int, int> b){
    // Nếu hoành độ khác nhau thì sắp cho nó tăng dần
    if (a.fi != b.fi)
        return a.fi < b.fi;
    // Nếu hoành độ bằng nhau thì sắp tung độ giảm dần
    return a.se > b.se;
}

int main(){
    int n;
    cin >> n;
    vector<pair<int, int>> v;
    for (int i = 0; i < n; i++)
    {
        int temp1, temp2;
        cin >> temp1 >> temp2;
        v.push_back({temp1, temp2});
    }
    QuickSort(v, 0, n - 1, comp);
    for (auto i : v)
        cout << i.fi << " " << i.se << '\n';
    return 0;
}

```

1.6. VU33_MaxStr

➤ Đề bài:

KHÓA SỐ

Để tăng độ an toàn chống hiện tượng cướp ngân hàng ngày càng phổ biến người ta dùng khóa số với mã mở khóa đơn giản nhưng rất hiệu quả. Trên cửa ra vào hiển thị một xâu khá dài các ký tự số. Các chữ số có thể di chuyển đổi chỗ cho nhau hoặc bị xóa. Muốn mở khóa người ta phải **di chuyển** các chữ số và trong trường hợp cần thiết – **xóa** vài chữ số để nhận được *xâu lớn nhất* thỏa mãn điều kiện đã cài đặt. Điều kiện này được thay đổi thường xuyên. Hôm nay điều kiện đó là "Số nhận được phải chia hết cho 3". Số nhận được có thể bắt đầu bằng các chữ số 0. Xâu "000" sẽ lớn hơn xâu "00".

Hãy xác định khóa mở cửa.

Dữ liệu: Vào từ thiết bị nhập chuẩn gồm một xâu ký tự số có độ dài lớn hơn 2 và không vượt quá 10^5 .

Kết quả: Đưa ra thiết bị xuất chuẩn xâu khóa mở cửa.

Ví dụ:

INPUT	OUTPUT
105	510

➤ Ý tưởng:

Rõ ràng, với 1 số bất kì để chia hết cho 3 và cách để xóa vài chữ số để nhận được thì ta cần xét tới tổng của các chữ số trong số đó. Do vậy, ta sẽ có các trường hợp như sau:

- Nếu tổng các chữ số chia hết cho 3, giữ nguyên
- Nếu tổng các chữ số chia hết cho 3 dư 1, xóa theo thứ tự sau và thỏa rồi thì dừng:
 - Xóa đi 1 chữ số nhỏ nhất chia 3 dư 1
 - Xóa đi 2 chữ số nhỏ nhất chia 3 dư 2
- Nếu tổng các chữ số chia hết cho 3 dư 2, xóa theo thứ tự sau và thỏa rồi thì dừng:
 - Xóa đi 1 chữ số nhỏ nhất chia 3 dư 1
 - Xóa đi 2 chữ số nhỏ nhất chia 3 dư 2

Sau đó, ta hãy sort lại toàn bộ string và in ra kết quả.

Nhìn lại thì cách trên chắc chắn là đúng và chính xác vì nếu trong trường hợp nó có dư giả như dư 1 thì: nếu phải xóa đi 3 chữ số trở lên cùng với việc không xóa số chia hết cho 3 vì như vậy là vô nghĩa thì ta sẽ có 3 trường hợp là:

- Cả 3 chữ số trong đó đều cùng số dư: 1 hoặc 2 → vô nghĩa vì tổng chia hết cho 3.
- Có 2 chữ số trong đó cùng số dư, như vậy có ít nhất 1 số còn lại khác số dư và khi ta ghép mỗi số của hai nhóm này thì được tổng chia hết cho 3 → vô nghĩa.
- Nếu có 4 chữ số trở lên, thì ta cứ lấy 3 trong số chúng sẽ thấy nó vô nghĩa do hai trường hợp trên.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
```

```

void printres(vector<int> &arr){
    for(int i = arr.size() - 1; i >= 0; i--) cout << arr[i];
}

void printpattern(int n){
    for(int i = 0; i < n; i++) cout << 0;
}

vector<int> splitDigits(string s){
    vector<int> digits;
    // Chuyển ký tự thành số
    for (char c : s) digits.push_back(c - '0');
    return digits;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    string s;
    cin >> s;
    vector<int> digits = splitDigits(s);
    int n = digits.size();
    sort(digits.begin(), digits.end());
    int sum = accumulate(digits.begin(), digits.end(), 0);
    vector<int> mod1, mod2;
    for (int d : digits) {
        if (d % 3 == 1) mod1.push_back(d);
        else if (d % 3 == 2) mod2.push_back(d);
    }
    if (sum % 3 == 0) {
        printres(digits);
        return 0;
    }
    if (sum % 3 == 1) {
        if (!mod1.empty()) {
            digits.erase(find(digits.begin(), digits.end(), mod1[0]));
            printres(digits);
            return 0;
        }
        if (mod2.size() >= 2) {
            digits.erase(find(digits.begin(), digits.end(), mod2[0]));
            digits.erase(find(digits.begin(), digits.end(), mod2[1]));
            printres(digits);
            return 0;
        }
    }
}

```

```

else { // sum % 3 == 2
    if (!mod2.empty()) {
        digits.erase(find(digits.begin(), digits.end(), mod2[0]));
        printres(digits);
        return 0;
    }
    if (mod1.size() >= 2) {
        digits.erase(find(digits.begin(), digits.end(), mod1[0]));
        digits.erase(find(digits.begin(), digits.end(), mod1[1]));
        printres(digits);
        return 0;
    }
}
printpattern(n);
return 0;
}

```

1.7. VQ44_FLOWERS(template)

➤ Đề bài:

VQ44. ĐÈN HOA

Tên chương trình: FLOWERS.???

Để chuẩn bị một ngày lễ lớn lần thứ k thành phố quyết định dùng đèn LED kết thành k bông hoa trang trí dọc đường phố chính, mỗi bông hoa được kết từ một loại bóng LED cùng màu. Để có hiệu ứng ánh sáng tốt nhất Công ty Chiếu sáng được yêu cầu tạo ra được càng nhiều hoa khác màu càng tốt. Trong kho của Công ty có n bộ đèn LED mỗi bộ lắp được một đèn và bộ thứ i có màu a_i .

Hãy chỉ ra k màu của các bộ đèn cần chọn sao cho số lượng hoa khác màu lắp được là nhiều nhất. Nếu có nhiều cách khác nhau thì đưa ra cách tùy chọn.

Dữ liệu: Vào từ file văn bản FLOWERS.INP:

- ✚ Dòng đầu tiên chứa 2 số nguyên n và k ($1 \leq k \leq n \leq 10^5$),
- ✚ Dòng thứ 2 chứa n số nguyên $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9, i = 1 \div n$).

Kết quả: Đưa ra file văn bản FLOWERS.OUT trên một dòng k số nguyên – màu của các đèn được chọn.

Ví dụ:

FLOWERS.INP
10 4
8 8 8 8 8 8 8 8 2 1

FLOWERS.OUT
1 2 8 8

➤ Ý tưởng:

Tương tự với bài này trong Assignment1 chỉ khác là ta phải tự viết hàm **sort**. Ta lấy tất cả mỗi cái của từng bộ phần tử khác nhau rồi sau đó lấy phần dư sót lại bù vào cho đủ.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<int> &arr, int low, int high) {
    int pivot = arr[low];
    int i = low - 1, j = high + 1;

    while (true){
        // Tìm phần tử bên trái ngoài cùng của dãy
        // mà lớn hơn hoặc bằng pivot
        do
        {
            i++;
        } while (arr[i] < pivot);
        // Tìm phần tử bên phải ngoài cùng của dãy
        // mà bé hơn hoặc bằng pivot
        do
        {
            j--;
        } while (arr[j] > pivot);
        /// Nếu hai phần tử đó gặp nhau
        if (i >= j)
            return j;
        // Nếu không thì sắp xếp cho đúng
        swap(arr[i], arr[j]);
    }
}

void QuickSort(vector<int> &arr, int low, int high){
    if (low < high)
        // pi là chỉ số pivot trước đã được
        // đặt ở đúng vị trí
        int pi = partition(arr, low, high);
        // Độc lập thực hiện sắp xếp cho hai phần
        // trái và phải của pivot như vậy
        QuickSort(arr, low, pi);
        QuickSort(arr, pi + 1, high);
    }
}
```

```

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n, k;
    cin >> n >> k;
    vector<int> a, res;
    for (int i = 0; i < n; i++)
    {
        int temp;
        cin >> temp;
        a.push_back(temp);
    }

    // Sắp xếp mảng theo thứ tự tăng dần
    QuickSort(a, 0, n - 1);

    // Khai báo vector after để đánh dấu biến đã đưa vào res
    vector<int> after = a;

    // Duyệt từ đầu tới cuối vector a để chọn các phần tử khác nhau
    res.push_back(a[0]);
    after[0] = 0;
    int temp1 = 0, temp2 = 0;
    for (int i = 1; i < n; i++){
        if (res[temp1] != a[i] && res.size() < k)
        {
            res.push_back(a[i]);
            after[i] = 0;
            temp1++;
        }
    }
    // Chọn phần tử có vị trí đầu tiên của vector after
    // chưa xuất hiện đến khi mảng res đủ
    while (res.size() < k) {
        if (after[temp2] != 0)
        {
            res.push_back(after[temp2]);
            after[temp2] = 0;
        }
        temp2++;
    }

    for (int j : res)
        cout << j << " ";
}

```

1.8. Kiểm kê

➤ Đề bài:

Kiểm kê

Cửa hàng X có nhiều loại hàng, mỗi loại hàng đều có mã khác nhau. Tuy nhiên, trong lúc bán hàng, nhân viên quên không nhập số lượng mỗi loại mà chỉ nhập mã các loại hàng trên từng dòng. Bạn hãy đếm xem có bao nhiêu loại hàng khác nhau?

Ví dụ: Có một số loại hàng có mã lần lượt là [123, 45, 8, 19, 1, 2333].

Bạn nhân viên sẽ nhập theo từng dòng [123, 45, 45, 8, 1, 123, 123, 8, 8].

Số loại hàng khác nhau trong dãy được nhập sẽ là 4.

Input:

- Dòng đầu tiên là số nguyên N ($1 \leq N \leq 5 \times 10^4$) là số lượng mã loại hàng được nhập.
- N dòng tiếp theo, mỗi dòng là một mã của loại hàng được nhập, độ dài của mã có thể lên tới 100.

Output:

- Một dòng chứa số nguyên là số các loại hàng khác nhau.

Input	Output
10	7
123	
123444	
123444	
12	
1	
8	
1	
455	
455	
9	

➤ Ý tưởng:

Cơ bản là ở đây ta chỉ sắp xếp lại các mã theo thứ tự rồi so sánh từng cặp liên tiếp để tìm ra số các loại hàng khác nhau.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<string> &arr, int low, int high)
{
    string pivot = arr[low];
    int i = low - 1, j = high + 1;

    while (true) {
        // Tìm phần tử bên trái ngoài cùng của dãy
        // mà lớn hơn hoặc bằng pivot
    }
}
```

```

        do
        {
            i++;
        } while (arr[i] < pivot);
        // Tìm phần tử bên phải ngoài cùng của dãy
        // mà bé hơn hoặc bằng pivot
        do
        {
            j--;
        } while (arr[j] > pivot);
        /// Nếu hai phần tử đó gặp nhau, không thì sắp xếp cho đúng
        if (i >= j)
            return j;
        swap(arr[i], arr[j]);
    }
}

void QuickSort(vector<string> &arr, int low, int high) {
    if (low < high) {
        // pi là chỉ số pivot trước đã được
        // đặt ở đúng vị trí
        int pi = partition(arr, low, high);
        // Độc lập thực hiện sắp xếp cho hai phần
        // trái và phải của pivot như vậy
        QuickSort(arr, low, pi);
        QuickSort(arr, pi + 1, high);
    }
}

int main() {
    int n, count = 1;
    cin >> n;
    vector<string> arr;
    for (int i = 0; i < n; i++) {
        string temp;
        cin >> temp;
        arr.push_back(temp);
    }
    QuickSort(arr, 0, n - 1);
    for (int i = 1; i < n; i++) {
        if (arr[i] != arr[i - 1]) {
            count++;
        }
    }
    cout << count;
    return 0;
}

```

1.9. MergeSort

➤ Đề bài:

Merge Sort

Bài toán: Thuật toán sắp xếp trộn

Tiếp theo, chúng ta sẽ thực hiện cài đặt thuật toán sắp xếp trộn (merge sort). Merge sort có độ phức tạp thời gian là $O(n \log(n))$, vì vậy nó được coi là một thuật toán hiệu quả hơn so với 3 thuật toán sắp xếp đã cài đặt trước đó. Merge sort là một thuật toán chia để trị (divide and conquer). Để minh họa cách hoạt động của nó, chúng ta sẽ cài đặt theo yêu cầu sau:

Yêu cầu: Dựa trên mã giả được cung cấp dưới đây, hãy cài đặt thuật toán sắp xếp trộn (merge sort) để sắp xếp lại một mảng có N phần tử. Trong quá trình chạy thuật toán, hãy in ra mảng A sau mỗi lần trộn hai mảng con.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 20$)
- Dòng tiếp theo chứa N số nguyên A_i là các phần tử của mảng

Output:

- Các dòng tiếp theo in ra cấu hình của mảng A .

Ví dụ:

	INPUT	OUTPUT
	[728 800] 703 671 628 625 518 508 392 331	
	[703 728 800] 671 628 625 518 508 392 331	
	703 728 800 [628 671] 625 518 508 392 331	
	[628 671 703 728 800] 625 518 508 392 331	
10	628 671 703 728 800 [518 625] 508 392 331	
800 728 703 671 628 625 518 508 392 331	628 671 703 728 800 [508 518 625] 392 331	
	628 671 703 728 800 508 518 625 [331 392]	
	628 671 703 728 800 [331 392 508 518 625]	
	[331 392 508 518 625 628 671 703 728 800]	

MERGE-SORT(A, p, r)

```
1 if  $p < r$ 
2    $q = \lfloor (p + r) / 2 \rfloor$ 
3   MERGE-SORT( $A, p, q$ )
4   MERGE-SORT( $A, q + 1, r$ )
5   MERGE( $A, p, q, r$ )
```

➤ Ý tưởng:

Đơn giản là mô phỏng lại thuật toán merge sort và in ra những thay đổi sau mỗi lần trộn.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

void merge(vector<int>& arr, int left, int mid, int right) {
    vector<int> left_part(arr.begin() + left, arr.begin() + mid + 1);
    vector<int> right_part(arr.begin() + mid + 1, arr.begin() + right + 1);

    int i = 0, j = 0, k = left;
    while (i < left_part.size() && j < right_part.size()) {
        if (left_part[i] <= right_part[j]) {
            arr[k++] = left_part[i++];
        } else {
            arr[k++] = right_part[j++];
        }
    }
}
```

```

while (i < left_part.size()) {
    arr[k++] = left_part[i++];
}
while (j < right_part.size()) {
    arr[k++] = right_part[j++];
}

// Định dạng chuẩn để in ra
for(int i = 0; i < arr.size(); i++){
    if(i == left && i == 0) cout << "[ " << arr[i];
    else if(i == 0) cout << arr[i];
    else if(i == left) cout << " [ " << arr[i];
    else if(i == arr.size() - 1 && i == right) cout << " " << arr[i] << "
]\n";
    else if(i == right) cout << " " << arr[i] << " ]";
    else if(i == arr.size() - 1) cout << " " << arr[i] << "\n";
    else cout << " " << arr[i];
}
}

void merge_sort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int N;
    cin >> N;
    vector<int> A(N);
    for (int i = 0; i < N; i++) {
        cin >> A[i];
    }
    merge_sort(A, 0, N - 1);
    return 0;
}

```

1.10. InsertionSort

➤ Đề bài:

Insertion Sort

Bài toán: Thuật toán sắp xếp chèn

Cũng như thuật toán sắp xếp nổi bọt, thuật toán sắp xếp chèn cũng là một trong những thuật toán đơn giản nhất.

Yêu cầu: Hãy cài đặt thuật toán sắp xếp chèn để **sắp xếp lại tăng dần** mảng có N phần tử. Trong lúc chạy thuật toán, nếu xảy ra hành động *swap* (đổi phần tử hoặc gán phần tử mới vào vị trí thích hợp), hãy in ra mảng sau hành động swap đó. Bằng cách này, các bạn có thể hình dung được cách hoạt động của thuật toán sắp xếp.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 200$)
- Dòng tiếp theo chứa N số nguyên A là các phần tử của mảng

Output:

- Các dòng tiếp theo in ra cấu hình của mảng A mỗi lần mảng A xảy ra hành động swap.

Ví dụ:

INPUT	OUTPUT
	83530 83530 48217 20995
	64425 83530 48217 20995
	64425 83530 83530 20995
	64425 64425 83530 20995
4	48217 64425 83530 20995
83530 64425 48217 20995	48217 64425 83530 83530
	48217 64425 64425 83530
	48217 48217 64425 83530
	20995 48217 64425 83530

Link tham khảo: [Insertion Sort Algorithm - GeeksforGeeks](https://www.geeksforgeeks.org/insertion-sort-algorithm/)

➤ Ý tưởng:

Đơn giản là mô phỏng lại thuật toán insertion sort và in ra thay đổi sau mỗi lần trộn.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

void InsertionSort(int arr[], int n){
    for (int i = 1; i < n; ++i){
        int k = arr[i];
        int j = i - 1;

        /* Chuyển các phần tử lớn hơn
        phần tử đang xét sang phải
        1 vị trí so với vị trí hiện tại */
        while (j >= 0 && arr[j] > k) {
            arr[j + 1] = arr[j];
            j--;
        }
        for(int l = 0; l < n; l++){
            cout << arr[l] << " ";
        }
        cout << "\n";
    }
}
```

```

        /* Chèn phần tử đang xét
        vào vị trí đúng */
        arr[j + 1] = k;
        for(int t = 0; t < n; t++){
            cout << arr[t] << " ";
        }
        cout << "\n";
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    InsertionSort(arr, n);

    return 0;
}

```

1.11. BubbleSort

➤ Đề bài:

Bubble Sort

Bài toán: Như vậy, sau khi đã cài đặt và chạy thử nghiệm trên *thuật toán nhị phân* trên mảng (đã được sắp xếp), chúng ta thấy được hiệu quả của mảng được sắp xếp trong việc tìm kiếm phần tử. Như vậy, sau khi học về giải thuật tìm kiếm, nội dung các thuật toán tiếp theo mà các bạn sẽ được học đó chính là các thuật toán về *sắp xếp (sorting)*.

Một trong những thuật toán sắp xếp cơ bản nhất đó chính là thuật toán *tìm kiếm nổi bọt (bubble sort)*. Ý tưởng của thuật toán nổi bọt (trong minh họa này là tăng dần) rất đơn giản. Cụ thể, cứ mỗi hai phần tử kế nhau sẽ được hoán chỗ (*swap*) nếu phần tử bên trái lớn hơn phần tử bên phải.

Yêu cầu: Hãy cài đặt thuật toán tìm kiếm nổi bọt để sắp xếp lại mảng có N phần tử. Trong lúc chạy thuật toán, nếu xảy ra hành động *swap*, hãy in ra mảng sau hành động swap đó. Bằng cách này, các bạn có thể hình dung được cách hoạt động của thuật toán sắp xếp.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 200$)
- Dòng tiếp theo chứa N số nguyên A là các phần tử của mảng

Output:

- Các dòng tiếp theo in ra cấu hình của mảng A mỗi lần mảng A xảy ra hành động swap.

Ví dụ:

INPUT	OUTPUT
4 49920 15837 14483 3601	15837 49920 14483 3601
	15837 14483 49920 3601
	15837 14483 3601 49920
	14483 15837 3601 49920
	14483 3601 15837 49920
	3601 14483 15837 49920

Link tham khảo: [Bubble Sort Algorithm - GeeksforGeeks](https://www.geeksforgeeks.org/bubble-sort-algorithm/)

➤ Ý tưởng:

Đơn giản là mô phỏng lại thuật toán insertion sort và in ra thay đổi sau mỗi lần trộn.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

void BubbleSort(vector<int> &arr)
{
    int n = arr.size();
    // Xét tất cả các cặp liên kề trong mảng

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            // Sắp xếp các cặp này
            if (arr[j] > arr[j + 1])
            {
                swap(arr[j], arr[j + 1]);
                for(int k = 0; k < n; k++){
                    cout << arr[k] << " ";
                }
                cout << "\n";
            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    vector<int> arr;
    for (int i = 0; i < n; i++) {
        int temp;
        cin >> temp;
        arr.push_back(temp);
    }
    BubbleSort(arr);
    return 0;
}
```

1.12. SelectionSort

➤ Đề bài:

Selection Sort

Bài toán: Thuật toán sắp xếp chọn

Thuật toán *selection sort* cũng là một trong những thuật toán đơn giản.

Yêu cầu: Hãy cài đặt thuật toán sắp *selection sort* để **sắp xếp lại tăng dần** mảng có N phần tử. Trong lúc chạy thuật toán, nếu xảy ra hành động *swap*, hãy in ra mảng sau hành động *swap* đó. Bằng cách này, các bạn có thể hình dung được cách hoạt động của thuật toán sắp xếp.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 200$)
- Dòng tiếp theo chứa N số nguyên A_i là các phần tử của mảng

Output:

- Các dòng tiếp theo in ra cấu hình của mảng A mỗi lần mảng A xảy ra hành động *swap*.

Ví dụ:

	INPUT	OUTPUT
	23466 90219 78622 71136 38905 95303	
6	23466 38905 78622 71136 90219 95303	
95303 90219 78622 71136 38905 23466	23466 38905 71136 78622 90219 95303	

Link tham khảo: [Selection Sort - GeeksforGeeks](https://www.geeksforgeeks.org/selection-sort/)

➤ Ý tưởng:

Đơn giản là mô phỏng lại thuật toán selection sort và in ra thay đổi sau mỗi lần trộn.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

void SelectionSort(int arr[], int n)
{
    // Xét từng vị trí trong mảng
    for (int i = 0; i < n - 1; i++)
    {
        int pos = i;
        /* Tìm phần tử nhỏ nhất
        từ vị trí i tới n */
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[pos])
                pos = j;
        }
    }
}
```

```

        /* Đưa phần tử thứ i
        về đúng vị trí */
        if (pos != i) { // Chỉ swap khi cần
            swap(arr[i], arr[pos]);
            for (int k = 0; k < n; k++) {
                cout << arr[k] << " ";
            }
            cout << "\n";
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    SelectionSort(arr, n);
    return 0;
}

```

2. Advance

2.1. MaxMinSum

- Tóm tắt đề bài: Tính tổng (Max – Min) của mọi dãy con có độ dài K trong mảng N
- Ý tưởng:

Ta sẽ tách làm 2 phần Max và Min ra và tính riêng biệt.

Với mỗi số a_i , ta sẽ tính có bao nhiêu dãy nhận a_i là Max và bao nhiêu nhận a_i là Min. Nếu ta sắp lại mảng N thì nó sẽ như sau:

- Số dãy con nhận a_i là Max: C_{i-1}^{K-1}
- Số dãy con nhận a_i là Min: C_{N-i}^{K-1}

Lúc này, đáp án sẽ là: $a_i C_{i-1}^{K-1} - a_i C_{N-i}^{K-1} \bmod p$, với $p = 10^9 + 7$.

Đối với phần tổ hợp thì tôi tham khảo theo tài liệu sau: [Cách tính nCk](#)

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N = 1e5 + 1;
const int MOD = 1e9 + 7;

int fact[N], ifact[N];

int binpow(int a, int b)
{
    if (b <= 0)
        return 1;
    if (b == 1)
        return a;

    int p = binpow(a, b / 2);

    if (b % 2 == 0)
        return p * p % MOD;

    return p * p % MOD * a % MOD;
}

// Chuẩn bị
void prepare()
{
    // Tính fact[]
    fact[0] = 1;
    for (int i = 1; i <= N; i++)
        fact[i] = fact[i - 1] * i % MOD;

    // Tính ifact[]
    ifact[N] = binpow(fact[N], MOD - 2);
    for (int i = N - 1; i >= 0; i--)
        ifact[i] = ifact[i + 1] * (i + 1) % MOD;
}

// Hàm tính nCk
int C(int n, int k)
{
    return fact[n] * ifact[k] % MOD * ifact[n - k] % MOD;
}
```

```

int32_t main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    prepare();
    int n, k, a[N];
    cin >> n >> k;
    int sum = 0;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    sort(a + 1, a + 1 + n);

    for (int i = k; i <= n; i++)
        sum = (sum + a[i] * C(i - 1, k - 1)) % MOD;
    for (int i = 1; i <= n - k + 1; i++)
        sum = ((sum - a[i] * C(n - i, k - 1)) % MOD + MOD) % MOD;
    cout << sum;
    return 0;
}

```

2.2. khangtd.Login1

- Tóm tắt đề bài: Cho những user và password tương ứng. In ra mật khẩu gần nhất.
- Ý tưởng: Sử dụng map<string, string> để lưu trữ user và password rồi gọi tới.
- Script:

```

#include <bits/stdc++.h>
using namespace std;
int main(){
    int n, k;
    string s1, s2, temp;
    cin >> n >> k;
    map<string, string> mp;
    for(int i = 0; i < n; i++){
        cin >> s1 >> s2;
        mp[s1] = s2;
    }
    for(int i = 0; i < k; i++){
        cin >> temp;
        if(mp.find(temp) != mp.end())
            cout << mp[temp] << '\n';
        else cout << "Chua Dang Ky!" << "\n";
    }
    return 0;
}

```

2.3. khangtd.Login2

- Tóm tắt đề bài: In ra tất cả các password của user đã được nhập.
- Ý tưởng: Thay map<string,string> thành map<string, vector<string>> để lưu trữ
- Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n, k;
    cin >> n >> k;
    map<string, vector<string>> mp;

    for(int i = 0; i < n; i++){
        string s1, s2;
        cin >> s1 >> s2;
        mp[s1].push_back(s2);
    }

    for(int i = 0; i < k; i++){
        string temp;
        cin >> temp;
        if(mp.find(temp) != mp.end()){
            for(string i : mp[temp]){
                cout << i << " ";
            }
            cout << "\n";
        }
        else cout << "Chua Dang Ky!" << "\n";
    }
    return 0;
}
```

2.4. khangtd.DetectVirus

- Tóm tắt đề bài: Nếu xâu T có trong xâu S thì in ra đáp án “YES” và vị trí xuất hiện của xâu T trong xâu s. Ngược lại, ghi ra “NO”.
- Ý tưởng: Vì xâu S và T nhỏ (chỉ dưới 1000 ký tự) nên ta có thể dùng phương pháp vét cạn để tìm ra đáp án đúng.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    string s, t;
    vector<int> pos;
    cin >> s >> t;
    int m = s.size(), n = t.size();
    if(n > m){
        cout << "NO";
        return 0;
    }
    for(int i = 0; i <= m - n; i++){
        if(s.substr(i, n) == t) pos.push_back(i);
    }
    if(pos.size() == 0){
        cout << "NO";
    }
    else{
        cout << "YES\n";
        for(int i : pos) cout << i + 1 << " ";
    }
    return 0;
}
```

2.5. khangtd.DetectVirus2

- Tóm tắt đề bài: Thay khoảng độ dài của hai xâu là dưới 10^6 so với bài trước.
- Ý tưởng: Sử dụng thuật toán [Knuth-Morris-Pratt \(KMP\)](#) để xử lý
- Script:

```
#include <bits/stdc++.h>
using namespace std;

vector<int> KMP(string &s, string &t) {
    int m = s.size(), n = t.size();
    vector<int> LPS(n, 0), pos;

    // Coding LPS
    int preLPS = 0, i = 1;
```

```

while(i < n){
    if (t[i] == t[preLPS]){
        LPS[i] = preLPS + 1;
        preLPS++;
        i++;
    }
    else if (preLPS == 0){
        LPS[i] = 0;
        i++;
    }
    else preLPS = LPS[preLPS - 1];
}
// Coding KMP
i = 0;
int j = 0;
while(i < m){
    if(s[i] == t[j]) i++, j++;
    else j ? j = LPS[j - 1] : i++;
    if(j == n) pos.push_back(i - j + 1);
}
return pos;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    string s, t;
    cin >> s >> t;

    if (t.size() > s.size()) {
        cout << "NO";
        return 0;
    }

    vector<int> pos = KMP(s, t);

    if (pos.empty()) cout << "NO\n";
    else {
        cout << "YES\n";
        for (int i : pos) cout << i << " ";
    }
    return 0;
}

```

2.6. Linear Search 4

➤ Đề bài:

Equal Diverse Teams

Bài toán:

Alice có N học sinh trong lớp, được đánh số từ 1 đến N . Sinh viên thứ i có chuyên môn về môn học A_i .

Alice phải chia học sinh thành hai đội. Tính duy nhất của một nhóm được định nghĩa là số lượng môn học riêng biệt sao cho có ít nhất một sinh viên trong nhóm có chuyên môn về môn học đó. Ví dụ: tính duy nhất của một nhóm được biểu diễn bằng $A = [1, 3, 1, 4, 4]$ là 3.

Alice muốn chia học sinh của lớp thành hai đội sao cho mỗi học sinh thuộc đúng một đội và điểm duy nhất của mỗi đội chính xác là K . Liệu anh ấy có làm được như vậy không?

Input :

- Dòng đầu tiên chứa số nguyên T là số bộ test. Mô tả của từng trường hợp thử nghiệm như sau:
- Dòng đầu tiên của mỗi test chứa hai số nguyên N và K
- Dòng thứ hai của mỗi test chứa N số nguyên $A_1, A_2, \dots, A_N$.

Ràng buộc:

- $1 \leq T \leq 10^5$
- $2 \leq N \leq 10^5$
- $1 \leq K \leq N$
- $1 \leq A_i \leq N$
- Tổng của N trên tất cả các test case không quá 3×10^5

Output:

- Đối với mỗi trường hợp thử nghiệm, in YES nếu Alice có thể tạo ra hai đội thỏa mãn các điều kiện đã cho, nếu không thì in NO.

➤ Ý tưởng:

Vì phải chia học sinh thành hai đội và điểm duy nhất của mỗi đội đúng bằng K nên khi gọi m là số lượng các môn học phân biệt mà các sinh viên có chuyên môn thì ta có hai trường hợp lớn sau:

- Nếu $m < k$ hoặc $m \geq 2k$ thì không thể chia đúng k môn ở mỗi nhóm.
- Nếu $k \leq m \leq 2k$ thì:
 - Nếu số giá trị xuất hiện nhiều hơn 1 lần mà $\geq 2k - m$ thì có cách để chia.
 - Ngược lại, thì không thể có cách chia.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    cin >> t;

    while (t--){
        int n, k, count = 0;
        cin >> n >> k;
        map<int, int> mp;

        for(int i = 0; i < n; i++){
            int temp;
            cin >> temp;
            mp[temp]++;
            if(mp[temp] == 2) count++;
        }

        int m = mp.size();
        if(m < k || m > 2*k) cout << "NO\n";
        else{
            if(m + count >= 2*k) cout << "YES\n";
            else cout << "NO\n";
        }
    }

    return 0;
}
```

2.7. Vượt mức Pickleball v2

➤ Đề bài:

VƯỢT MỨC PICKLEBALL (PHIÊN BẢN DSA)

Sau khi khóa đầu tiên của lớp Pickleball kết thúc, giáo sư Pickleball PMQ lại mở lớp học chơi Pickleball cho các bạn UIT và USSH. Vào kỳ thi cuối kỳ, mỗi bạn được đánh bóng n lần, với lần thứ i được chấm x điểm. Trong quá trình đánh bóng, một cú đánh bóng được xem là vượt mức Pickleball nếu điểm của cú đánh đó lớn hơn hoặc bằng 2 lần giá trị điểm trung vị của d lần đánh trước đó (*Lưu ý: Nếu giá trị d chẵn, giá trị trung vị được xác định là giá trị trung bình của 2 phần tử ở giữa*).

Cho n điểm đánh bóng của một bạn sinh viên bất kỳ, bạn hãy viết chương trình kiểm tra xem bạn đó đã vượt mức Pickleball bao nhiêu lần?

INPUT

- Dòng đầu tiên là 2 số nguyên dương n và d ($1 \leq n \leq 2 \times 10^5$; $1 \leq d \leq n$)
- Dòng tiếp theo chứa n số $a_1, a_2, \dots, a_N$ với ($0 \leq a_i \leq 200$) là điểm số mà sinh viên đó đạt được trong n lần đánh bóng.

OUTPUT

- Số lần sinh viên đó vượt mức Pickleball.

➤ Ý tưởng:

Chú ý là $a_i \in [0, 200]$, với mọi i và từ đây ta gọi mảng đếm `count()` để đếm số lần xuất hiện của các phần tử trong khoảng d được xét. Ta có thể tìm trung vị theo 2 trường hợp:

- Nếu d lẻ, tìm x nhỏ nhất sao cho $\text{sum}[i]$ là tổng của các giá trị của hàm đếm tính với i chạy từ 0 tới x mà $\text{sum}[i] \geq \lceil \frac{d}{2} \rceil$ và trả về x .
- Nếu d chẵn, tìm y nhỏ nhất sao cho $\text{sum}[i]$ là tổng của các giá trị của hàm đếm tính với i chạy từ 0 tới y mà $\text{sum}[i] \geq \lceil \frac{d}{2} \rceil + 1$ và trả về $\frac{x+y}{2}$, với x từ phần tính trên.

➤ Script:

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 1e6 + 5;
int a[N];
int count[N];

double median(int d){
    double ans = 0 , sum = 0;
    for(int i = 0 ; i <= 200 ; i++){
        sum += count[i];
        if(sum >= (d + 1) / 2){
            ans = i;
            break;
        }
    }

    if(d % 2 == 1)
        return ans;

    sum = 0;
    for(int i = 0 ; i <= 200 ; i++){
        sum += count[i];
        if(sum >= (d + 1) / 2 + 1){
            ans = (ans + i) / 2.0;
            break;
        }
    }

    return ans;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n , d;
    cin >> n >> d;

    for(int i = 1 ; i <= n ; i++)
        cin >> a[i];
```

```

int i = 1, j = 1, res = 0;
while(i <= n){
    while(i - j > d){
        count[a[j]]--;
        j++;
    }

    if(i > d && median(d) * 2 <= a[i]){
        res++;
    }
    count[a[i]]++;
    i++;
}
cout << res;
return 0;
}

```

2.8. Bốn ông điền

- Tóm tắt đề bài: Tìm số lần đổi chỗ 2 phần tử ít nhất sao cho mảng có thứ tự.
- Ý tưởng:

Đây là một bài toán liên quan đến đồ thị và tìm số cạnh cần thêm hoặc bớt để tạo thành một dãy tăng hoặc giảm. Ta sẽ sử dụng DFS (Depth-First Search) để đếm số chu trình trong đồ thị được xây dựng từ mảng đầu vào, từ đó tính toán kết quả. Sau đây là các bước chi tiết:

- Xây dựng đồ thị dựa trên thứ tự tăng dần và giảm dần.
- Mỗi thành phần liên thông trong đồ thị đại diện cho một chu trình hoặc đường.
- Số cạnh cần thêm/bớt = Tổng (số đỉnh trong mỗi thành phần - 1).
- So sánh hai trường hợp rồi lấy giá trị nhỏ nhất.

- Script:

```

#include <bits/stdc++.h>
using namespace std;

const int N = 1e6 + 9;

int a[N];
bool b[N];
pair<int, int> p[N];
vector<int> G[N];

int DFS(int u){

```

```

        b[u] = true;
        for (auto v : G[u]) {
            if (b[v])
                continue;
            return DFS(v) + 1;
        }
        return 1;
    }
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i], p[i] = {a[i], i};

    sort(p + 1, p + 1 + n);
    for (int i = 1; i <= n; i++)
        G[i].push_back(p[i].second);
    int res = 0;
    for (int i = 1; i <= n; i++)
    {
        if (b[i])
            continue;
        res += DFS(i) - 1;
    }

    sort(p + 1, p + 1 + n, greater<pair<int, int>>());
    for (int i = 1; i <= n; i++)
        G[i].clear(), b[i] = false;
    for (int i = 1; i <= n; i++)
        G[i].push_back(p[i].second);
    int res2 = 0;
    for (int i = 1; i <= n; i++){
        if (b[i])
            continue;
        res2 += DFS(i) - 1;
    }

    cout << min(res, res2);
    return 0;
}

```

2.9. Huấn luyện chuột

- Tóm tắt đề bài: Cho số nguyên tố p và N chú chuột, mỗi con chuột có thể lấy thẻ từ 1 tới N . Hãy đếm số cách các chú chuột lấy thẻ sao cho các tấm thẻ tạo thành dãy không giảm theo module p .
- Ý tưởng:

Gọi số chú chuột lấy tấm thẻ giá trị x_i là i . Khi đó, ứng với mỗi cách sắp xếp là 1 lần chọn cho từng tấm thẻ tức là nghiệm phương trình sau: $x_1 + x_2 + \dots + x_N = N$, $x_i \geq 0$

Rõ ràng, đây là bài toán chia kẹo Euler với đáp án là: C_{n+k-1}^{k-1} . Mà vì N có giới hạn đến 10^{12} nên ta sẽ sử dụng [định lý Lucas](#) để tính tổ hợp và đây là lời giải.

- Script:

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int MAX = 1e5 + 1;
int MOD = 1e9 + 7;

int binpow(int a, int b)
{
    if (b <= 0)
        return 1;
    if (b == 1)
        return a;

    int p = binpow(a, b / 2);

    if (b % 2 == 0)
        return p * p % MOD;
    return p * p % MOD * a % MOD;
}

// Chuẩn bị
const int LIM = 1e5;
int fact[MAX], ifact[MAX];

void prepare()
{
    // Tính fact[]
    fact[0] = 1;
    for (int i = 1; i <= LIM; i++)
        fact[i] = fact[i - 1] * i % MOD;
```

```

// Tính ifact[]
ifact[MOD - 1] = binpow(fact[MOD - 1], MOD - 2);
for (int i = MOD - 2; i >= 0; i--)
    ifact[i] = (ifact[i + 1] * (i + 1)) % MOD;
}

// Hàm tính C
int C(int n, int k)
{
    if (k > n) return 0;
    return fact[n] * ifact[k] % MOD * ifact[n - k] % MOD;
}

int32_t main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int t, n;
    cin >> t >> MOD;
    prepare();

    while(t--){

        cin >> n;
        int K = n - 1;
        int N = 2*n - 1;
        int res = 1;

        while(N > 0){
            int N1 = N % MOD;
            int K1 = K % MOD;

            N /= MOD;
            K /= MOD;

            res = res * C(N1, K1) % MOD;
        }

        cout << res << "\n";
    }

    return 0;
}

```